CS482/495/496 Software Project:
Poker Face Project

Abbas Daniah, David Ogunbanjo, Oreoluwa Adebusoye, Philip McCaffrey

2026-11-09

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name: Dr Hoang Bui
- Client title:
- Client email address:
- Client employer:
- How you know the client:

2 Project Description

2.1 Overview

[Add a few paragraphs describing your project succinctly. What problem are you trying to solve, what is the purpose of your project? Why does your client want this project?]

2.2 Key Features

[At this point you should have a basic understanding of your client's needs. List out the key features of the software system the client wants you to build.]

2.3 Why this Project is Interesting

[Why did you decide this project was interesting enough to you to be a capstone project? What about this project is enticing? Why should anyone care?]

2.4 Areas of CS required

[What subfields of computer science seem most likely to be relevant to your project? A capstone must involve multiple.]

2.5 Potential Concerns and Questions

[Is there any aspect of this project that makes you unsure if it will work, either due to your own interests/background, or that you aren't sure if it fits the requirements? Are there questions you have about this project that you want instructor feedback about?]

2.6 Summary of Efforts to Find a Project

(Not necessary for 482) [Briefly list out when/how you've discussed with this client, and if you've discussed with other clients who either didn't work out or didn't respond. If you considered a different project and it didn't work out, why didn't it work out?]

[Most CS495 projects end here. The sections below are for CS482 and CS496 software projects].

2.7 Comparison to Draft

[For CS496 only, focus on highlighting the major differences between the draft proposal in CS495 and this one here. If there are no major differences, you can remove this subsection.]

3 Requirements

3.1 Non-Functional Requirements

[Non-functional requirements are just as important as functional requirements. Dont forget to specify them.]

Table 1 presents the NFRs for this software project.

ID	NFR Title	Category	Description
NFR1	NFR Example 1	Usability	Description of the NFR (it does not follow a user story template)
NFR2	NFR Example 2	Security	Description of the NFR (it does not follow a user story template)

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

[In CS482, all functional requirements are written as User Stories. In CS496, some projects may use a different template to write the requirements. The table below is an example of writing the Stories. Adapt accordingly to different templates or if you want to display more info.]

Table 2 presents the functional requirements for this software project.

ID	Story Title	Points	Description
S1	Story Example 1	5	As a user, I want to write a user story example, so that people will understand them.
S2	Story Example 2	2	As a user, I want to write a user story example, so that people will understand them.
	Total:	80	

Table 2: Functional requirements as User Stories.

3.3 AI Analysis on Functional Requirements

We used OpenAI's ChatGPT (GPT-6) to review the functional requirements for Poker Face. We asked the LLM to act as a senior software engineer and identify missing or unclear requirements in our user stories, particularly those related to gameplay, account management, table management, and player interactions.

The prompt used was:

> Act as a senior software engineer reviewing the functional requirements for Poker Face, a web-based seven-card stud poker application. Review the user stories and identify missing, unclear,

or hidden functional requirements. For each suggestion, explain why it matters and whether it is essential or optional.

The AI identified several requirements worth considering. One was handling situations where a player disconnects or refreshes the page during a game. We agreed that this was important because players should not lose their progress due to a temporary connection issue. Another suggestion was to clearly define how winners are determined and how ties are handled. We considered this essential because it directly affects gameplay.

The AI also suggested clarifying whether players could join games already in progress. We had already decided that players could only join tables with a Waiting status, so this did not require an additional user story.

Other suggestions included password reset functionality, leaderboards, and more detailed player statistics. We considered password reset and leaderboards reasonable additions because they support account management and player engagement. However, detailed player statistics were considered a lower priority.

Not all suggestions were appropriate for our project. For example, real-money betting and voice or video chat would add unnecessary complexity and were outside the scope of Poker Face. We decided not to include these features.

Overall, the AI analysis was useful for identifying requirements that were easy to overlook, particularly edge cases and situations that might occur during gameplay. However, we also realized that some suggestions were either already covered by our requirements or were not necessary for the application. Rather than accepting every suggestion, we evaluated each one based on our project goals, available time, and the importance of the feature.

4 System Design

4.1 Architecture

We use the Model-View-Controller (MVC) architectural pattern to separate the user interface, application logic, and data management. This separation allows different components of the system to be developed, tested, and maintained independently.

The View layer handles user interactions and displays information. The Controller layer processes requests, validates input, and coordinates application operations. The Model layer defines the structure of the data and handles database operations through Data Access Objects (DAOs).

Our main modules include User, Table, Game, Bot, and Chat. The User module manages accounts, profiles, friends, and administrative permissions. The Table module manages poker tables and their configurations. The Game module handles gameplay, betting, and winner determination. The Bot module supports computer-controlled players, while the Chat module handles player communication.

This modular structure allows team members to work on different features while maintaining a consistent organization throughout the application.

4.2 Diagrams

4.2.1 Multiplayer Betting Action Sequence Diagram

Figure 1 illustrates the sequence of interactions when a player performs a betting action during a multiplayer Seven-Card Stud game.

The player submits an action through the React frontend, which sends a request to the GameController. The controller validates the request, retrieves the game and table information, and verifies that the player belongs to the table.

The BettingService then processes the action within a MongoDB transaction. It validates the player's turn, checks the betting rules, updates the player's chip balance when necessary, and saves the updated game state. Afterward, the BotService processes any applicable bot turns, and the controller returns the updated game information to the frontend.

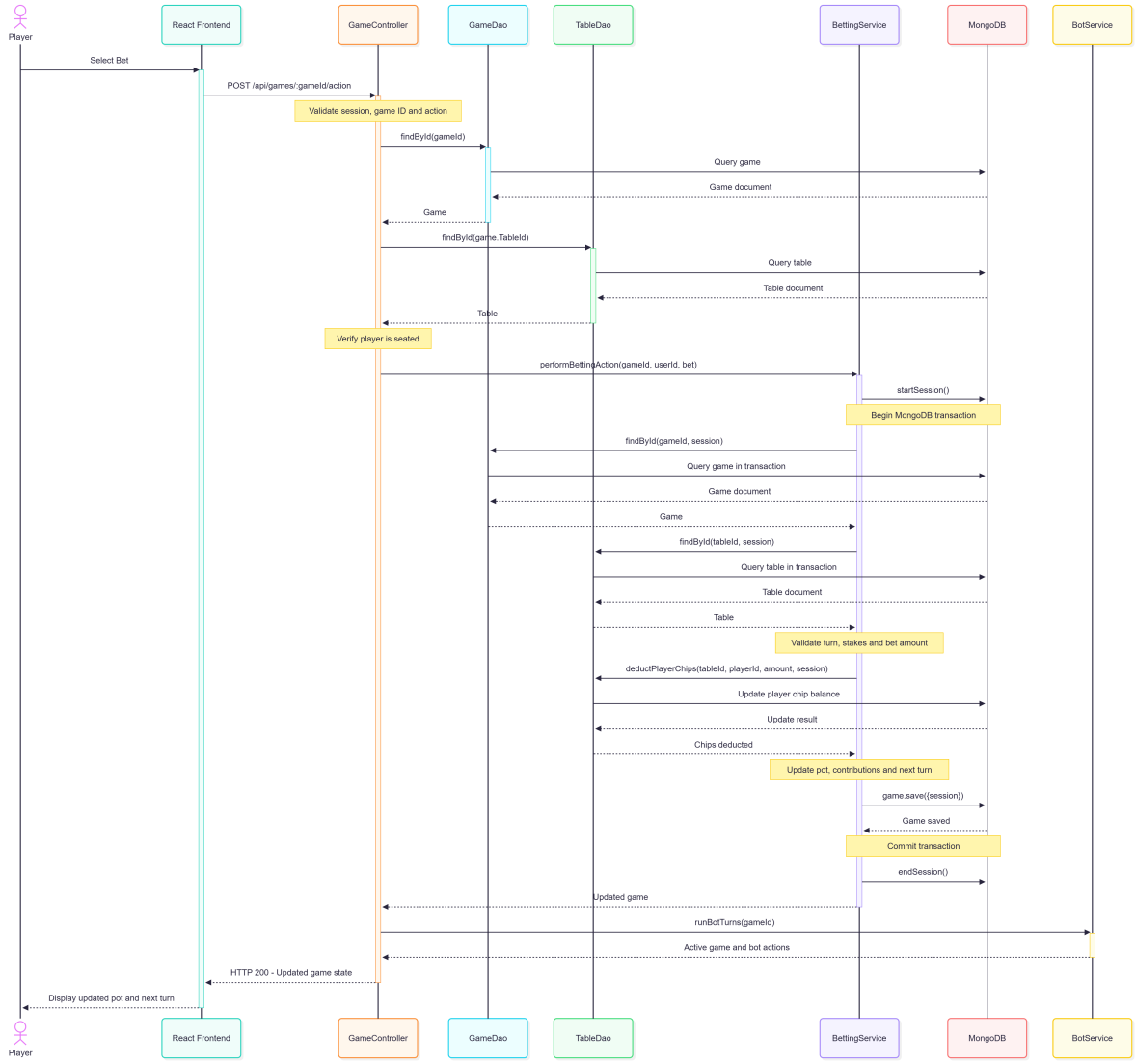


Figure 1: Sequence diagram for a multiplayer betting action in Poker Face

4.3 Architecture

We use the Model-View-Controller (MVC) architectural pattern to separate the user interface, application logic, and data management. This separation allows different components of the system to be developed, tested, and maintained independently.

The View layer handles user interactions and displays information. The Controller layer processes requests, validates input, and coordinates application operations. The Model layer defines the structure of the data and handles database operations through Data Access Objects (DAOs).

Our main modules include User, Table, Game, Bot, and Chat. The User module manages accounts, profiles, friends, and administrative permissions. The Table module manages poker tables and their configurations. The Game module handles gameplay, betting, and winner determination. The Bot module supports computer-controlled players, while the Chat module handles player communication.

This modular structure allows team members to work on different features while maintaining a consistent organization throughout the application.

4.4 Coding Standards

We will use camelCase for variables and functions, PascalCase for React components and classes, and UPPER_SNAKE_CASE for constants. Our MongoDB collections will also use PascalCase.

We will keep our code formatting consistent and use meaningful names for variables and functions. Comments will be used to explain complex logic when needed.

Each team member will work on separate Git branches, and changes will be reviewed before merging into the main branch. We will use Jest for testing and aim for at least 60% code coverage before merging. At the end of each iteration, completed and reviewed feature branches will be merged into the main branch to ensure that the iteration deliverable is available for evaluation.

4.5 Data

We will use MongoDB, a NoSQL database, to store our application data. Our main collections will be User, Table, Game, and ChatHistory.

```
User {
    _id: ObjectId
    UserName: string
    Email: string
    PasswordHash: string
    Chips: int
    AdsWatched: int
    Wins: int
    Losses: int
    GamesPlayed: int
    DateJoined: Date
    Avatar: Blob
    IsAdmin: bool
    FriendIDs: ObjectId array
}


Table {
    _id: ObjectId
    Name: string
    JoinCode: string
    Capacity: int
    UserIds: ObjectId array
    BotCount: int
    IsPublic: bool
    Level: string
    Status: string
    CurrentGameID: ObjectId
}

Contact {
    _id: ObjectId
    Name: string
    Email: string
    Subject: string
    Message: string
    Status: string
    createdAt: Date
```

```
    updatedAt: Date
}

Game {
    _id: ObjectId
    TableID: ObjectId
    PlayerStates: array
    CurrentTurn: ObjectId
    BettingRound: int
    Pot: int
    CurrentBet: int
    Status: string
}

ChatHistory {
    _id: ObjectId
    TableID: ObjectId (optional)
    UserIDs: ObjectId array
    Messages: array [{
        SenderID: ObjectId
        Words: string
        Timestamp: Date
    }]
}
```

4.6 UI Mocks

4.6.1 Homepage

The homepage introduces the Poker Face platform and provides options to log in, register, or play as a guest.



Figure 2: Poker Face Homepage

4.6.2 Login and Registration

The login and registration pages allow users to create an account, sign in, or continue playing as a guest.



Figure 3: Login and Registration Interface

4.6.3 Lobby

The lobby displays available poker tables and allows players to create, join, or observe games.



Figure 4: Poker Face Lobby

4.6.4 Table Creation

The table creation interface allows players to configure a table, including its level, capacity, public or private status, and number of bots.

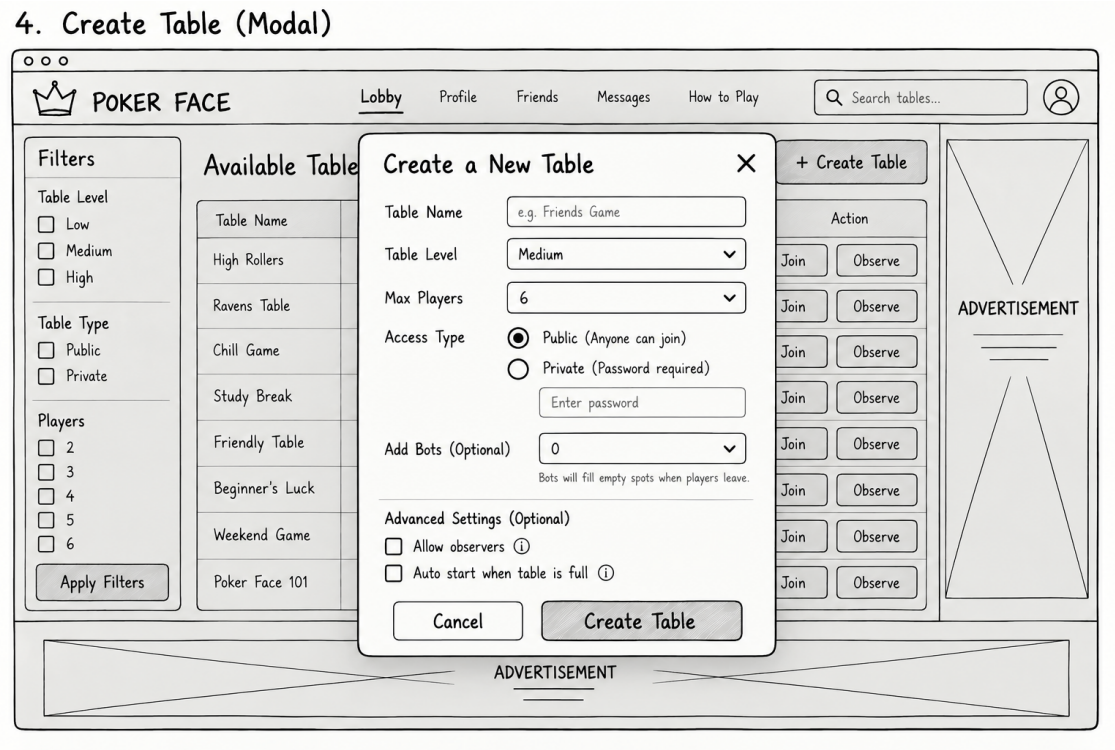


Figure 5: Create Table Interface

4.6.5 Poker Gameplay

The gameplay interface displays the poker table, player cards, chip balances, and betting controls. Players can also communicate through in-game chat.

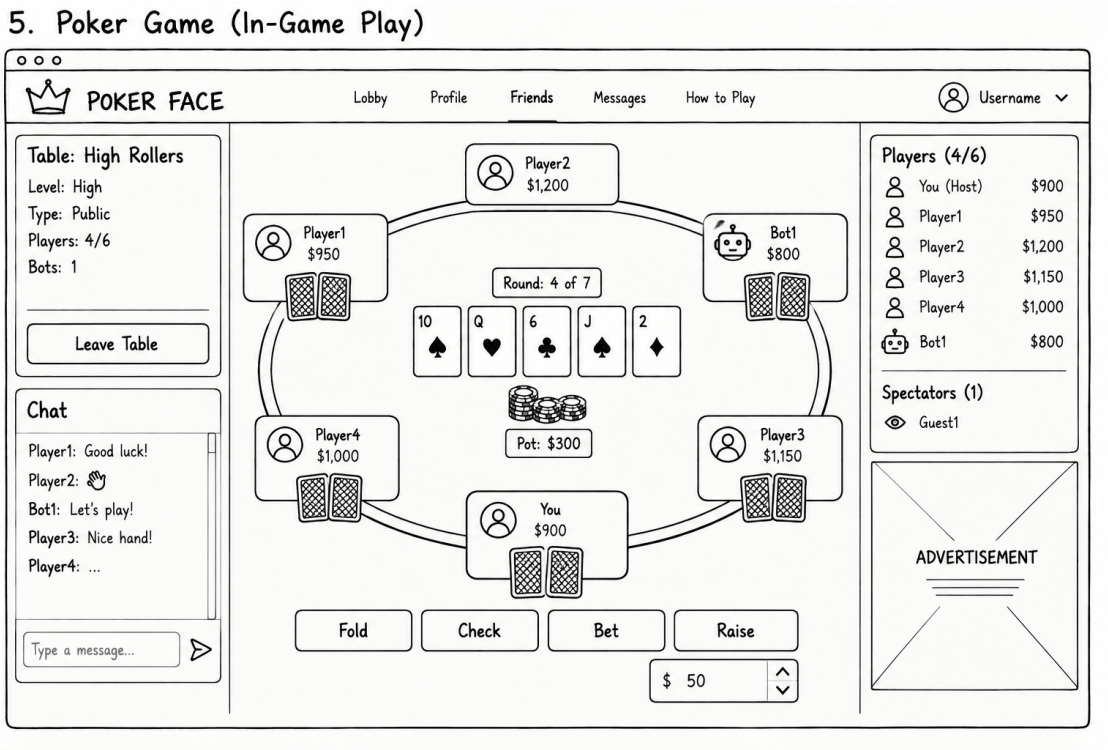


Figure 6: Seven-Card Stud Gameplay Interface

4.6.6 Game Results

The results interface displays the winning player, hand rankings, and chips won or lost after each hand.

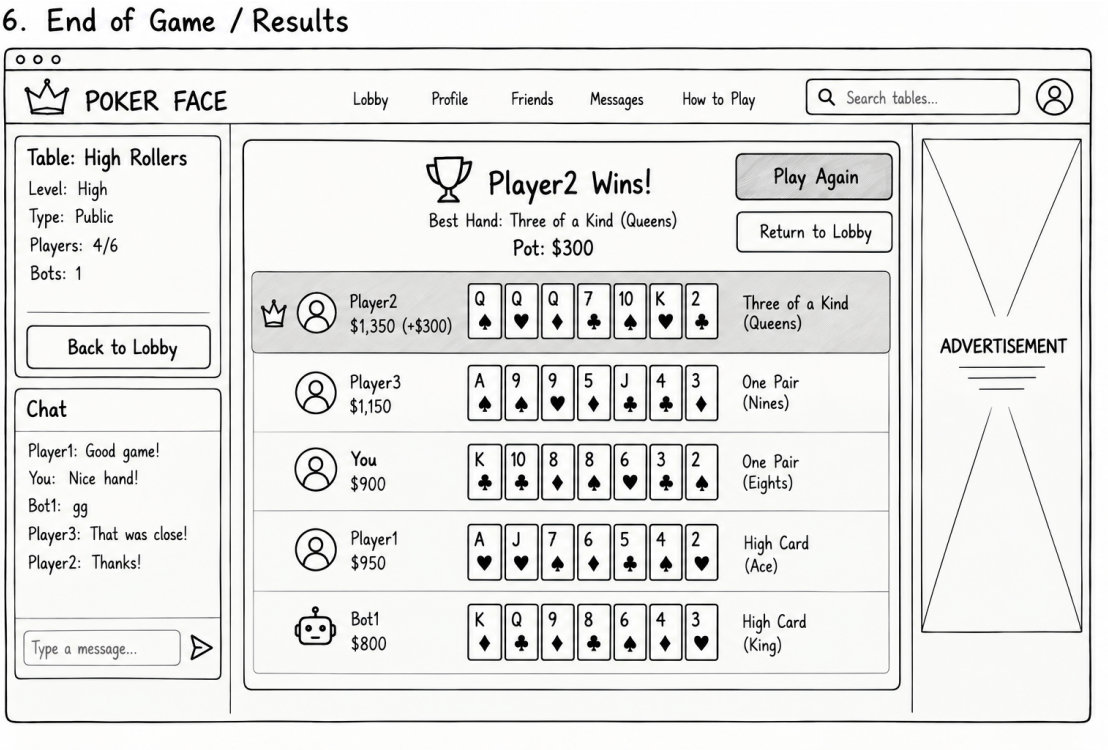


Figure 7: Game Results Interface

4.6.7 Game Tutorial

The tutorial page explains the rules of Seven-Card Stud, including card dealing, betting rounds, and poker hand rankings.

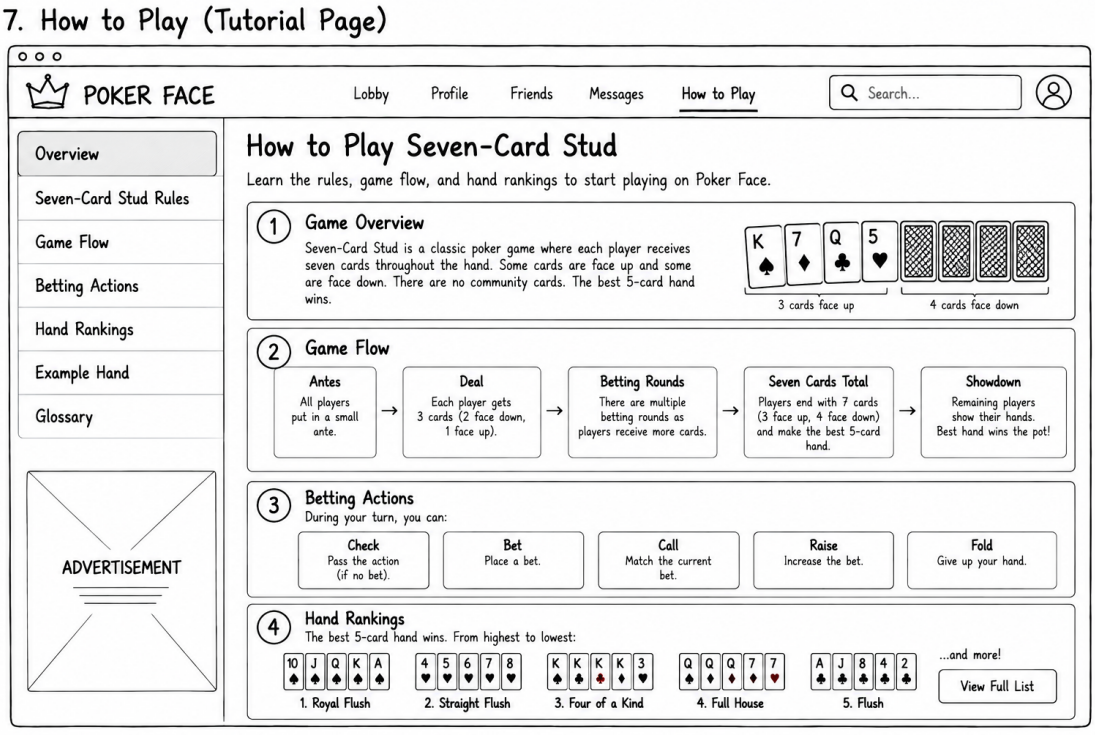


Figure 8: How to Play Tutorial

4.6.8 Player Profile

The player profile displays the user's avatar, chip balance, and gameplay statistics. Registered users can also update their profile information.

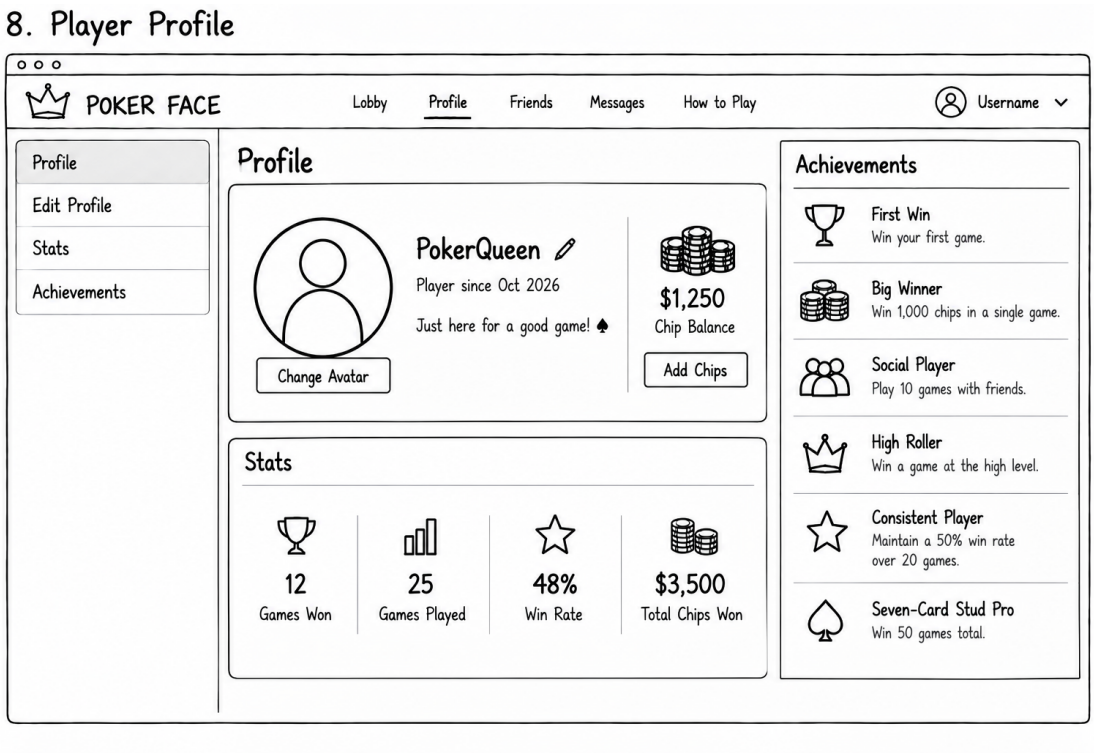


Figure 9: Player Profile Interface

4.6.9 Friends and Messaging

The friends and messaging interface allows registered users to search for players, manage friend requests, and exchange private messages.

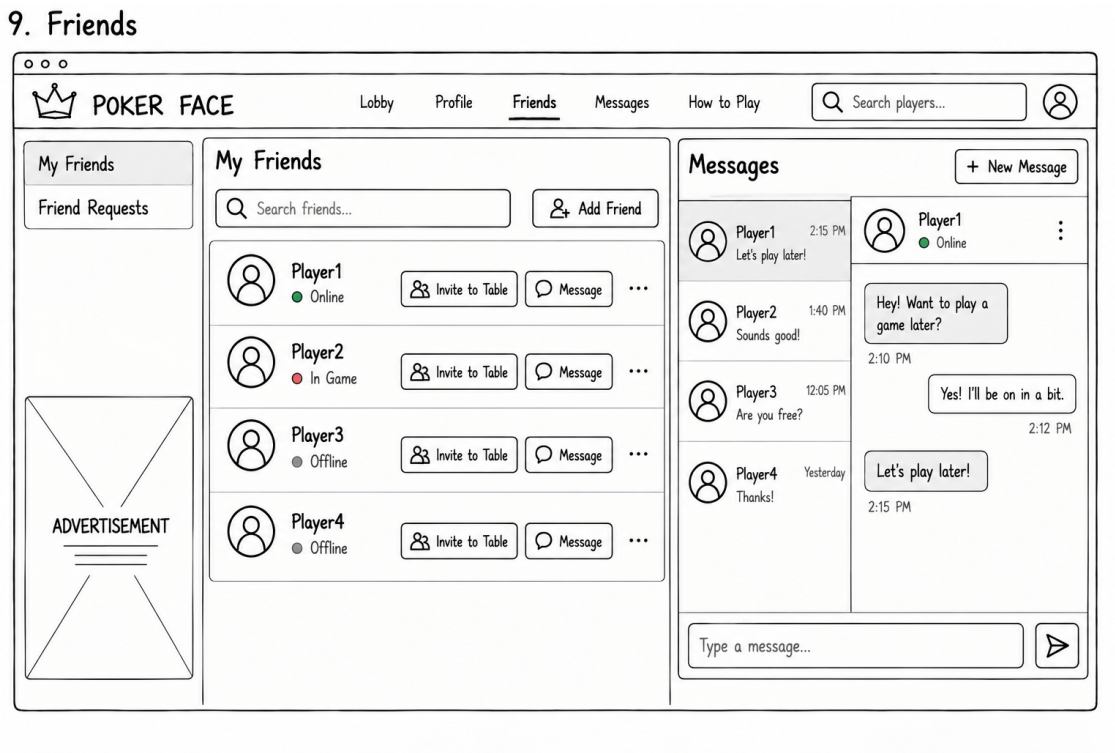


Figure 10: Friends and Messaging Interface

4.6.10 Admin Dashboard

The admin dashboard allows authorized administrators to view registered users, monitor games, and review player statistics and advertising engagement.

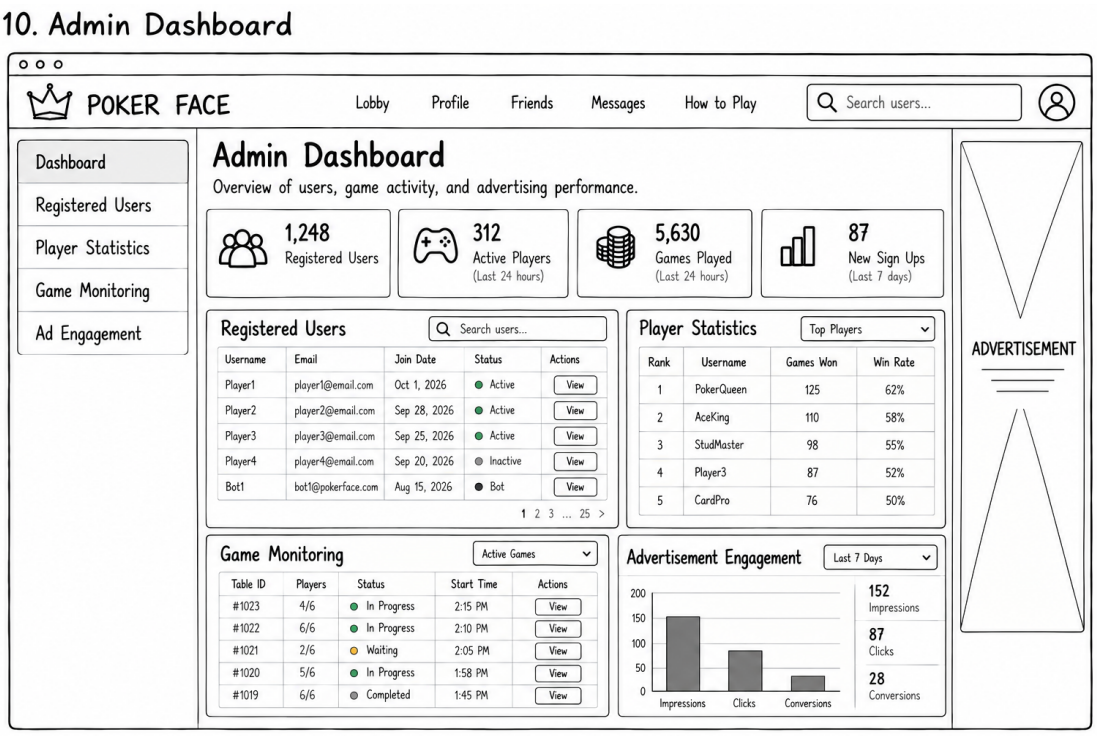


Figure 11: Administrator Dashboard

5 Iterations

5.1 Iteration Planning

[In CS496, you plan all iterations beforehand. In CS482, you update the planning here at each iteration.]

Table 3 shows the iteration planning.

			Points	
It.	**Dates**	**Stories**	**Planned**	**Done**
1	10/08 - 10/22	P4, P47, P48, P27, P34, P9, P19, P3, P8, P44, P45, P35, P1, P13, P29, P5	26	22
2	10/22 - 11/05	P48, P43, A33, A6, P15, P37, P40, P16, P49, P42, P2, P18, P20, P13, P29, P36, P14, P0, P32, P31	44	TBD
3	11/05 - 11/19	P7, P17, P52, P53, P54; additional stories to be determined	TBD	TBD
4	04/01 - 05/01	S12 Story Title, S13 Story Title, S14 Story Title, S15 Story Title	19	19
5	05/01 - 06/01	S16 Story Title, S17 Story Title	06	06
		Total:	70	65

Table 3: Iteration Planning for Incremental Deliveries

5.2 Iteration/Sprint 1

5.2.1 Planning

For Iteration 1, our team focused on developing the foundational features of Poker Face, including account management, player interactions, poker table management, and the player lobby.

We divided the user stories among the four team members based on the features they would be responsible for implementing.

Daniah was assigned player account creation (P4), account deletion (P47), account updates (P48), login and logout (P27), and profile viewing (P34).

Oreoluwa was assigned player lobby tables (P9), friend requests (P19), friend lookup (P3), and contacting an administrator (P8).

David was assigned viewing the friend list (P44), removing friends (P45), and choosing an avatar (P35).

Philip was assigned table creation (P1), exiting a table (P13), inactive table removal (P29), and earning chips (P5).

The goal of this iteration was to establish the core functionality needed for users to interact with the application before implementing the complete poker gameplay system in later iterations.

5.2.2 Work Done

During Iteration 1, we completed most of the user stories planned for the sprint. We worked on user accounts, profiles, friends, and table management, which are some of the main features needed before implementing the actual poker gameplay.

Daniah worked on account creation (P4), account deletion (P47), account updates (P48), login and logout (P27), and viewing player profiles (P34). These features allow users to register, log into their accounts, update their information, and manage their profiles.

Oreoluwa worked on the player lobby (P9), friend lookup (P3), friend requests (P19), and administrator contact form (P8). The lobby allows players to browse, search, and filter public tables, while the friend features allow players to find other users and send requests. The contact form allows users to submit messages that are stored in MongoDB.

David worked on viewing the friends list (P44), removing friends (P45), and choosing an avatar (P35). These features allow players to manage their friendships and personalize their profiles.

Philip worked on creating tables (P1), exiting tables (P13), removing inactive tables (P29), and earning chips (P5). This work focused on setting up and managing poker tables and supporting the chip system.

We also spent time connecting the frontend and backend, working with MongoDB, resolving merge conflicts, and writing automated tests. During testing, we found and fixed issues in the code and added more tests to improve our overall coverage.

Most of the planned stories were completed, although some parts still need further refinement and integration in the next iteration.

5.2.3 Testing Coverage

Automated testing was conducted using Jest to verify the backend functionality developed during Iteration 1. The tests covered user account creation and management, authentication, friend-related operations, table management, and administrator contact form submissions.

The team tested both successful operations and error-handling scenarios, including missing input fields, duplicate accounts, invalid login credentials, database errors, and unsuccessful requests. Additional tests were written for the data access objects (DAOs) to verify their interactions with Mongoose and MongoDB. Password hashing and verification were also tested using bcrypt.

The last confirmed coverage report showed 80.96% statement coverage, 67.40% branch coverage, 81.13% function coverage, and 81.65% line coverage. At that point, all 92 tests across 9 test suites passed successfully.

We consider this coverage satisfactory for Iteration 1 because it exceeds the required 80% coverage threshold for statements, functions, and lines, as well as the 50% branch coverage requirement.

However, some areas of the application still have lower coverage than others, particularly certain controller functions and database operations. In the next iteration, we plan to continue adding tests as new features are

implemented, with particular attention to error handling, edge cases, and integration between application components.

Figure 12 shows the test coverage

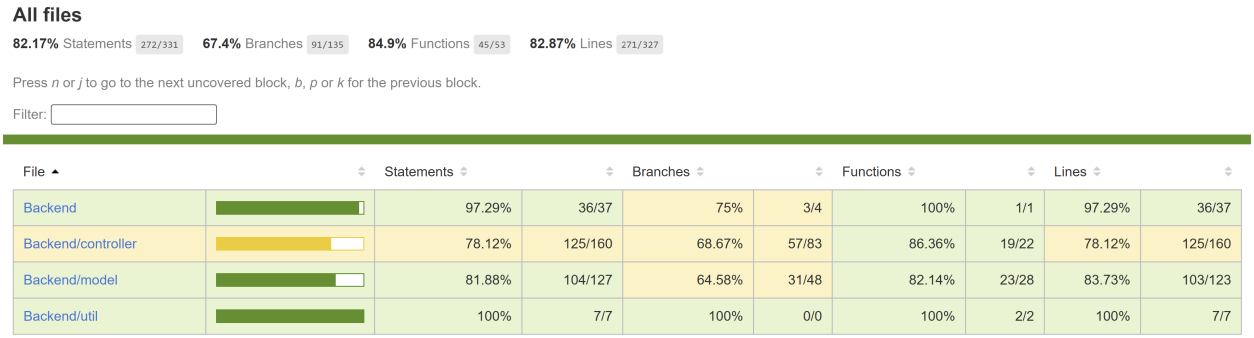


Figure 12: Iteration 1 test coverage report

5.2.4 Retroespective & Reflection

One of the challenges we faced during Iteration 1 was planning which stories to work on. We initially assigned stories without fully considering how some depended on others being completed first. As we started working, we realized that certain stories could not be implemented independently, so we had to drop some from the iteration and reassign others. This meant adjusting our original sprint plan as we went along.

We also underestimated how long some tasks would take and could have started working on them earlier. Some features required more work than we expected, especially when it came to connecting the frontend and backend, working with the database, and testing. As a result, we had more work left toward the end of the iteration than we anticipated.

Another challenge was testing and integrating everyone's work. We ran into issues with some functions and found that certain parts of the code had low test coverage. We had to spend additional time fixing these issues and writing more tests.

For the next iteration, we want to be more careful when selecting and assigning stories. We plan to identify dependencies before finalizing the sprint plan so that we do not assign stories that cannot be completed yet. We also want to start earlier, make more realistic estimates, and break larger tasks into smaller steps. Testing and merging changes throughout the iteration should also help us avoid having too much left to do near the deadline.

5.3 Iteration/Sprint 2

5.3.1 Planning

For Iteration 2, our team shifted focus toward implementing core poker gameplay mechanics, expanding administrative capabilities, and enhancing player automation and table interactions. Building on the foundational features from Iteration 1, we divided the user stories among the four team members based on their designated feature areas:

Daniah was assigned system administration and account management enhancements, taking ownership of player password resets (P43), finalizing player account updates (P48), implementing the admin dashboard (A33), and managing advertisements (A6).

Oreoluwa was assigned the core poker rules engine and turn-based mechanics, focusing on card distribution and viewing (P15, P37, P40), player betting actions like bets, raises, checks, and folds (P16), as well as turn and betting round progression (P49) and ante/bring-in mechanics (P42).

David was assigned player automation, social messaging, and user onboarding, leading the development of the player bot (P2), player private messaging (P18), and the interactive game tutorial (P20).

Philip was assigned table access and automated session management, completing the carryover work for exiting tables (P13) and removing inactive tables (P29), while implementing joining tables (P36), observing games (P14), guest player access (P0), idle timeouts (P31), AFK kicks (P32), and researching turn timer spike handling.

The goal of this iteration was to transition Poker Face from a basic user and lobby management framework into a playable poker ecosystem with functional game loops, automated bots, and robust table controls.

5.3.2 Work Done

During Iteration 2, we made significant progress toward making Poker Face a playable Seven-Card Stud application. Most of the planned features were implemented, including the core gameplay mechanics, bot functionality, messaging, and administrative features.

Oreoluwa implemented the main gameplay logic, including the initial card distribution, dealing cards for subsequent streets, and allowing players to view their cards. She also implemented betting actions such as folding, checking, calling, betting, and raising, along with ante and bring-in mechanics. This involved managing player chip contributions, the pot, turn order, and progression between betting rounds.

Daniah implemented password recovery and continued working on account management. She also developed the administrative dashboard and advertisement management functionality, expanding the features available to administrators.

David implemented computer-controlled players, allowing bots to participate in poker games and perform actions during their turns. He also worked on private messaging between players and the interactive tutorial to help new users understand the game.

Philip worked on table management and player access features, including joining and exiting tables, guest access, observing games, and handling inactive players and tables. The completion status of these features will be confirmed during the iteration acceptance review.

We also continued improving our automated testing. Additional Jest tests were written for gameplay, controllers, database operations, and messaging. In particular, we improved GameDao and TableDao coverage, bringing overall backend statement coverage to 90.87%.

5.3.3 Testing Coverage

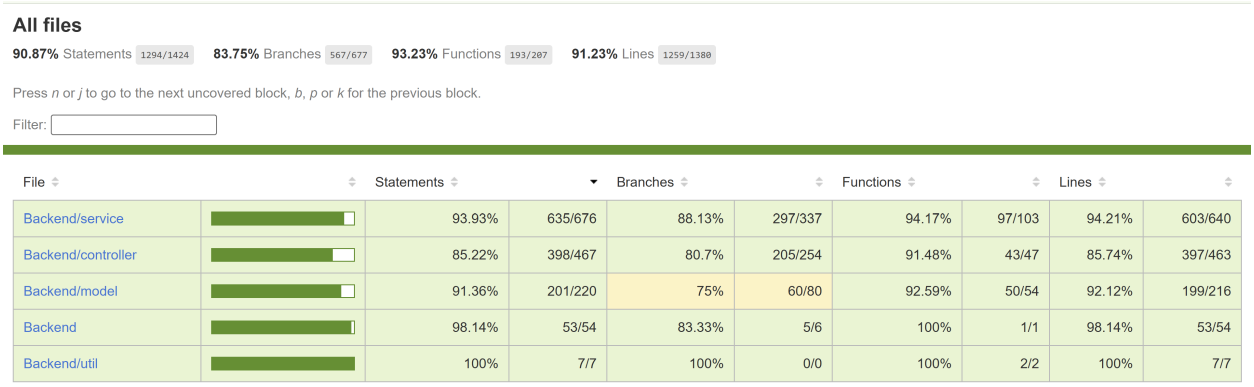
During Iteration 2, we continued using Jest to test the backend functionality of Poker Face. As we added the core gameplay features, we expanded our tests to cover card dealing, betting actions, game progression, database operations, and other backend services.

We also focused on improving coverage in areas that were not being tested enough. In particular, we added more tests for GameDao and TableDao, including game creation, retrieving games, advancing streets, table management, and chip deductions. This brought GameDao to 100% coverage across all four metrics and TableDao to 100% statement, function, and line coverage, with 83.33% branch coverage.

Our latest coverage report showed 90.87% statement coverage, 83.75% branch coverage, 93.23% function coverage, and 91.23% line coverage. This is an improvement from our previous results and exceeds the required coverage thresholds for the assignment.

We consider this coverage satisfactory because most of the backend functionality is now tested, including important gameplay operations and database interactions. However, there are still some areas with lower coverage, particularly certain controllers and error-handling branches. In the next iteration, we plan to continue adding tests as we integrate more features and focus on scenarios that are not yet covered.

Figure 13 shows our Iteration 2 test coverage report.



File	Statements		Branches		Functions		Lines	
Backend/service	93.93%	635/676	88.13%	297/337	94.17%	97/103	94.21%	603/640
Backend/controller	85.22%	398/467	80.7%	205/254	91.48%	43/47	85.74%	397/463
Backend/model	91.36%	201/220	75%	60/80	92.59%	50/54	92.12%	199/216
Backend	98.14%	53/54	83.33%	5/6	100%	1/1	98.14%	53/54
Backend/util	100%	7/7	100%	0/0	100%	2/2	100%	7/7

Figure 13: Iteration 2 backend test coverage report

5.3.4 Retrospective & Reflection

Team Reflection

Iteration 2 was more challenging than Iteration 1 because we started implementing the actual poker gameplay. Many of the features depended on one another, especially card dealing, betting, turn progression, and bot actions. This made it important for us to coordinate our work and make sure the different components could work together.

One challenge was managing these dependencies while working on separate features. Some functionality needed other parts of the game to be completed before it could be fully tested. We also spent additional time testing and fixing issues as the gameplay features came together.

For Iteration 3, we want to improve how we plan and coordinate dependent tasks. We should identify dependencies earlier, communicate more frequently about changes that affect other modules, and test features together throughout the iteration rather than waiting until the end. This should make integration easier and give us more time to address unexpected issues.

Oreoluwa's Personal Reflection

Personally, I learned a lot from implementing the core gameplay mechanics, especially betting actions and turn progression. One thing I found challenging was making sure the different betting rules worked correctly together. For example, a player's action can affect their chip balance, the pot, whose turn comes next, and whether the game should move to another betting round.

I also gained a better understanding of MongoDB transactions and why they are useful when multiple database updates need to happen together. Another thing I learned was the importance of reviewing test coverage rather than just checking whether tests pass. When I looked at our coverage report, I noticed that GameDao and TableDao had low coverage, so I added more tests for their database operations. This helped improve our overall coverage and gave me more confidence in those components.

For the next iteration, I want to spend more time testing edge cases as I implement features instead of going back to them later. I also want to focus more on testing complete gameplay scenarios to make sure the different parts of the game work correctly together.

5.4 Iteration/Sprint 3

5.4.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.4.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.4.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.4.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.5 Iteration/Sprint 4

[CS496 has 5 sprints. CS482 only has only 3 sprints (remove Iterations 4 and 5 from this doc if you are writing a doc for 482]

5.5.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.5.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.5.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.5.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.6 Iteration/Sprint 5

5.6.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.6.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.6.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.6.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project. Be concrete about your progress, you know how many story points your software is, how many points you completed (this shows your progress). You also how many points your team delivers at each iteration, therefore you can estimate how many more iterations it would take to finish the leftover points (show the math).]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently? How can you do better work in future projects? You may write this as a team or per person (or both — if all your iterations were team reflections, then it would be better to write individual reflections here)]

Appendix

[Appendix section if needed]